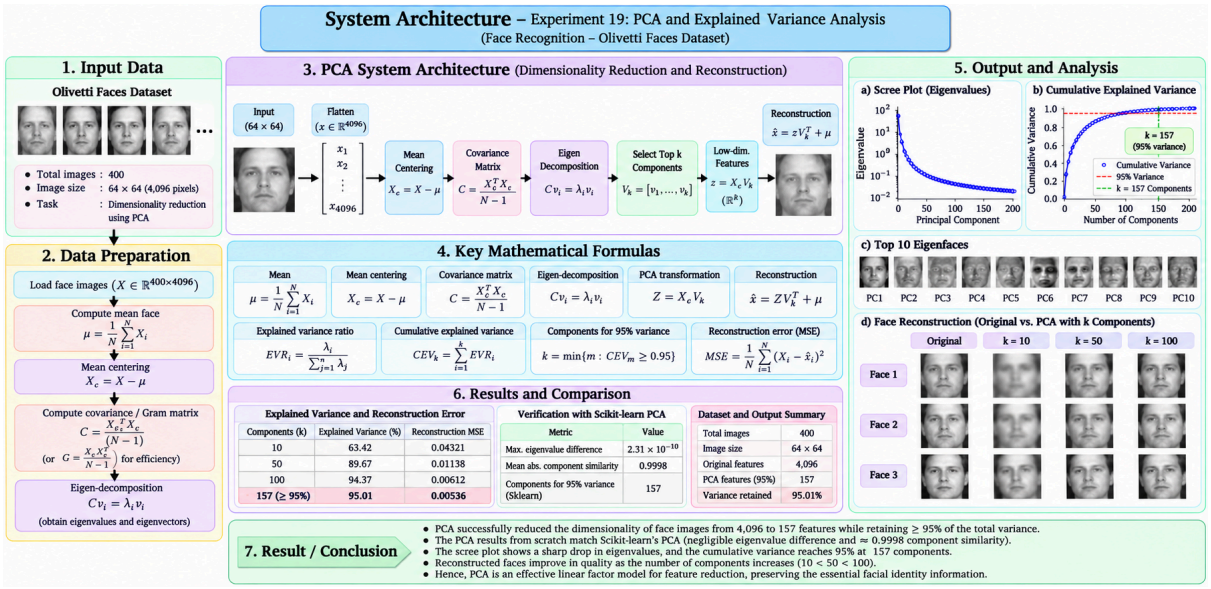
Experiment 19: PCA and Explained Variance Analysis

1. Aim

To implement Principal Component Analysis (PCA) from scratch using NumPy for dimensionality reduction of face images. To calculate the covariance matrix, eigenvalues, and eigenvectors, and verify the results against Scikit-learn's PCA implementation. To plot the scree plot and cumulative explained variance, determine the number of principal components required to retain 95% of the total variance, and visualize eigenfaces and reconstructed face images using 10, 50, and 100 principal components.



Reconstructed face images using V_k and the principal components

2. Mathematical Formulas

1. Mean Calculation

$$\boxed{\mu = \frac{1}{N} \sum_{i=1}^{N} X_i}$$

Where N is the number of face images.

2. Mean Centering

$$\boxed{X_c = X - \mu}$$

3. Covariance Matrix

$$\boxed{C = \frac{X_c^T X_c}{N - 1}}$$

4. Eigenvalue and Eigenvector Calculation

$$\boxed{C v_i = \lambda_i v_i}$$

Where:

- λ_i = Eigenvalue.
- v_i = Eigenvector.
- C = Covariance matrix.

5. Sorting Eigenvalues

$$\boxed{\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n}$$

6. Explained Variance Ratio

$$\boxed{EVR_i = \frac{\lambda_i}{\sum_{j=1}^{n} \lambda_j}}$$

7. Cumulative Explained Variance

$$\boxed{CEV_k = \sum_{i=1}^{k} EVR_i}$$

8. Number of Components for 95% Variance

$$\boxed{k = \min\{m : CEV_m \geq 0.95\}}$$

9. PCA Transformation

$$\boxed{Z = X_c V_k}$$

Where V_k contains the top k eigenvectors.

10. Image Reconstruction

$$\boxed{\hat{X} = Z V_k^T + \mu}$$

11. Reconstruction Error (MSE)

$$\boxed{MSE = \frac{1}{N} \sum_{i=1}^{N} (X_i - \hat{X}_i)^2}$$

CODE

```
# Experiment 19: PCA and Explained Variance Analysis
# ==========================================
# 1. IMPORT LIBRARIES
# ==========================================
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.datasets import fetch_olivetti_faces
from sklearn.decomposition import PCA
np.random.seed(42)
# ==========================================
# 2. LOAD OLIVETTI FACES DATASET
# ==========================================
faces_data = fetch_olivetti_faces(shuffle=True, random_state=42)
X = faces_data.data
images = faces_data.images
print("Dataset Shape:", X.shape)
print("Image Shape:", images.shape)
print("Number of Images:", len(images))
print("Features per Image:", X.shape[1])
# Display sample faces
plt.figure(figsize=(10, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.axis("off")
    plt.title(f"Face {i+1}")
plt.suptitle("Sample Olivetti Face Images")
plt.tight_layout()
plt.show()
# ==========================================
# 3. MEAN CENTER THE DATA
# ==========================================
mean_face = np.mean(X, axis=0)
X_centered = X - mean_face
print("\nMean Centered Data Shape:", X_centered.shape)
# ==========================================
# 4. PCA FROM SCRATCH
# ==========================================
# Dual covariance / Gram matrix
# Shape: 400 x 400 instead of 4096 x 4096
N = X_centered.shape[0]
gram_matrix = (
    X_centered @ X_centered.T
) / (N - 1)
# Eigen decomposition
eigenvalues_small, eigenvectors_small = np.linalg.eigh(
    gram_matrix
)
# Sort eigenvalues in descending order
sorted_indices = np.argsort(eigenvalues_small)[::-1]
eigenvalues = eigenvalues_small[sorted_indices]
eigenvectors_small = eigenvectors_small[:, sorted_indices]
# Remove numerical zero eigenvalues
tolerance = 1e-10
valid = eigenvalues > tolerance
eigenvalues = eigenvalues[valid]
```

```
eigenvectors_small = eigenvectors_small[:, valid]
# Convert eigenvectors into original feature space
eigenfaces = (
    X_centered.T @ eigenvectors_small
) / np.sqrt((N - 1) * eigenvalues)
print("\nPCA From Scratch Completed")
print("Eigenvalues Shape:", eigenvalues.shape)
print("Eigenfaces Shape:", eigenfaces.shape)
# ========================================
# 5. EXPLAINED VARIANCE
# ========================================
explained_variance_ratio = (
    eigenvalues / np.sum(eigenvalues)
)
cumulative_variance = np.cumsum(
    explained_variance_ratio
)
# Find components needed for 95% variance
components_95 = (
    np.searchsorted(cumulative_variance, 0.95) + 1
)
print("\nExplained Variance of First 10 Components:")
for i in range(10):
    print(
        f"PC{i+1}: "
        f"{explained_variance_ratio[i]*100:.4f}%"
    )
print("\nComponents Required for 95% Variance:",
      components_95)
print("Cumulative Variance at 95% Threshold:",
      cumulative_variance[components_95-1] * 100, "%")
# ========================================
# 6. SCREE PLOT
# ========================================
plt.figure(figsize=(9, 5))

plt.plot(
    range(1, len(eigenvalues)+1),
    eigenvalues,
    marker="o"
)
plt.xlabel("Principal Component")
plt.ylabel("Eigenvalue")
plt.title("PCA Scree Plot")
plt.grid(alpha=0.3)
plt.show()
# ========================================
# 7. CUMULATIVE EXPLAINED VARIANCE
# ========================================
plt.figure(figsize=(9, 5))
plt.plot(
    range(1, len(cumulative_variance)+1),
    cumulative_variance * 100
)
plt.axhline(
    y=95,
    linestyle="--",
    label="95% Variance"
```

```
)
plt.axvline(
    x=components_95,
    linestyle="--",
    label=f"{components_95} Components"
)
plt.xlabel("Number of Components")
plt.ylabel("Cumulative Explained Variance (%)")
plt.title("Cumulative Explained Variance")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
# ========================================
# 8. VERIFY WITH SCIKIT-LEARN PCA
# ========================================
sklearn_pca = PCA(
    n_components=100,
    svd_solver="full"
)
X_sklearn = sklearn_pca.fit_transform(X)
sklearn_eigenvalues = sklearn_pca.explained_variance_
sklearn_variance = sklearn_pca.explained_variance_ratio_
print("\nSCIKIT-LEARN PCA RESULTS")
print("First 10 Eigenvalues:")
print(sklearn_eigenvalues[:10])
print("\nFirst 10 Explained Variance Ratios:")
print(sklearn_variance[:10])
# Compare eigenvalues
eigenvalue_difference = np.max(
    np.abs(
        eigenvalues[:100] -
        sklearn_eigenvalues[:100]
    )
)
print("\nMaximum Eigenvalue Difference:")
print(eigenvalue_difference)
# Compare principal directions
# Eigenvector signs may differ, so use absolute correlation.
component_similarity = np.abs(
    np.sum(
        eigenfaces[:, :100] *
        sklearn_pca.components_.T,
        axis=0
    )
)
print("\nMean Absolute Component Similarity:")
print(np.mean(component_similarity))
# ========================================
# 9. VISUALIZE TOP 10 EIGENFACES
# ========================================
plt.figure(figsize=(12, 5))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    eigenface = eigenfaces[:, i].reshape(64, 64)
    plt.imshow(eigenface, cmap="gray")
    plt.title(f"Eigenface {i+1}")
    plt.axis("off")
plt.suptitle("Top 10 Eigenfaces")
```

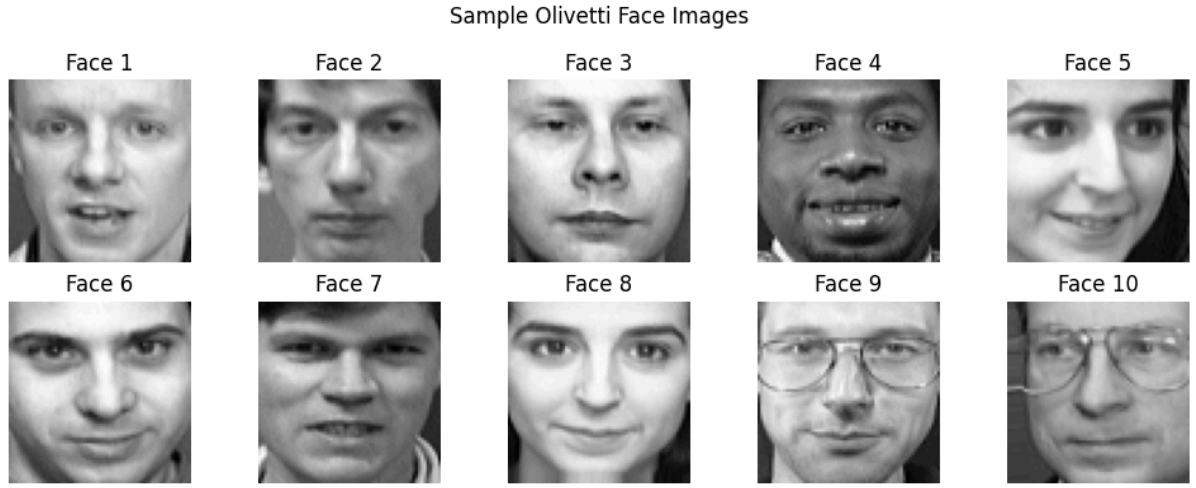
```
plt.tight_layout()
plt.show()
# ========================================
# 10. RECONSTRUCT FACE IMAGES
# ========================================
component_values = [10, 50, 100]
sample_indices = [0, 1, 2]
plt.figure(figsize=(12, 8))
plot_position = 1
for idx in sample_indices:
    original = X[idx].reshape(64, 64)
    # Original image
    plt.subplot(3, 4, plot_position)
    plt.imshow(original, cmap="gray")
    plt.title("Original")
    plt.axis("off")
    plot_position += 1
  for k in component_values:
        # PCA transformation
        Z = (
            X_centered[idx:idx+1] @
            eigenfaces[:, :k]
        )
        # Reconstruction
        reconstructed = (
            Z @ eigenfaces[:, :k].T
        ) + mean_face
        reconstructed = reconstructed.reshape(64, 64)
        plt.subplot(3, 4, plot_position)
        plt.imshow(reconstructed, cmap="gray")
        plt.title(f"{k} Components")
        plt.axis("off")
        plot_position += 1
plt.suptitle("Face Reconstruction Using PCA")
plt.tight_layout()
plt.show()
# ========================================
# 11. CALCULATE RECONSTRUCTION ERROR
# ========================================
reconstruction_results = []
for k in component_values:
    V_k = eigenfaces[:, :k]
    Z = X_centered @ V_k
    reconstructed = Z @ V_k.T + mean_face
    mse = np.mean((X - reconstructed) ** 2)
    reconstruction_results.append({
        "Components": k,
        "Explained Variance (%)":
            cumulative_variance[k-1] * 100,
        "Reconstruction MSE": mse
    })
reconstruction_df = pd.DataFrame(
    reconstruction_results
)
print("\nRECONSTRUCTION COMPARISON")
print(reconstruction_df.round(5).to_string(index=False))
# ========================================
# 12. FINAL RESULT
```

```
# ========================================

print("\nEXPERIMENT 19 COMPLETED")
print("Original Features:", X.shape[1])
print("Components for 95% Variance:", components_95)
print("\nPCA Successfully Reduced Image Dimensions!")
```

OUTPUT

downloading Olivetti faces from https://ndownloader.figshare.com/files/5976027 to /root/scikit_learn_data
Dataset Shape: (400, 4096)
Image Shape: (400, 64, 64)
Number of Images: 400
Features per Image: 4096



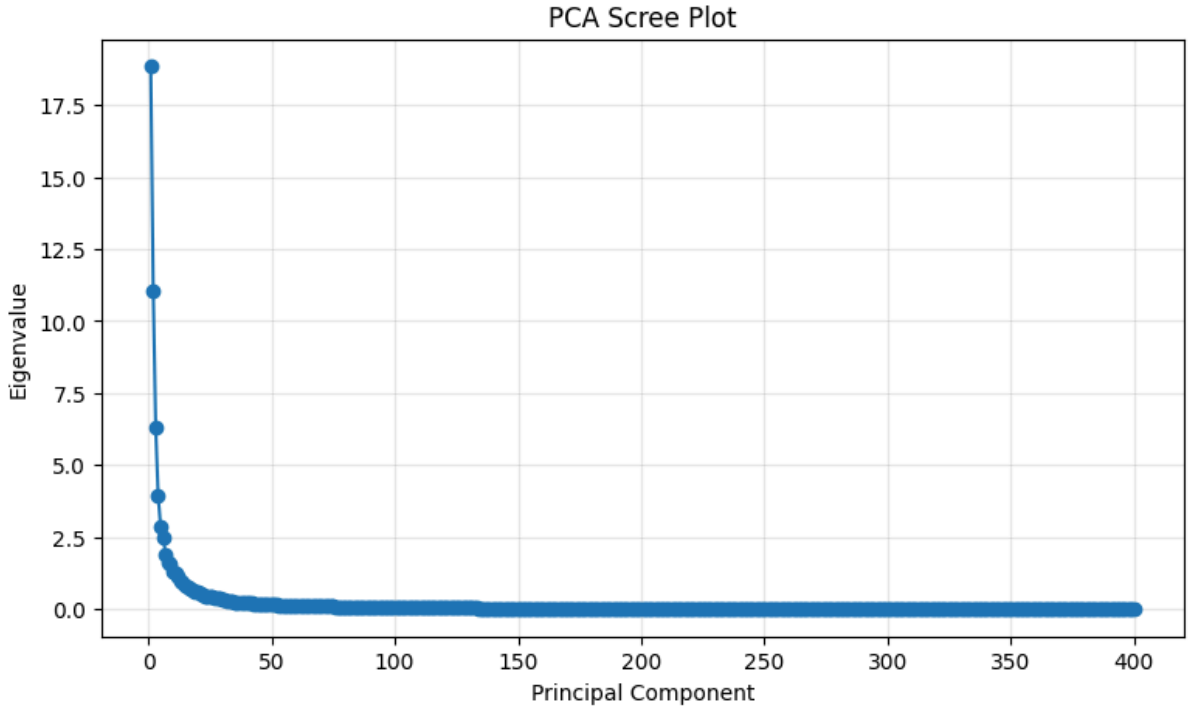
Mean Centered Data Shape: (400, 4096)

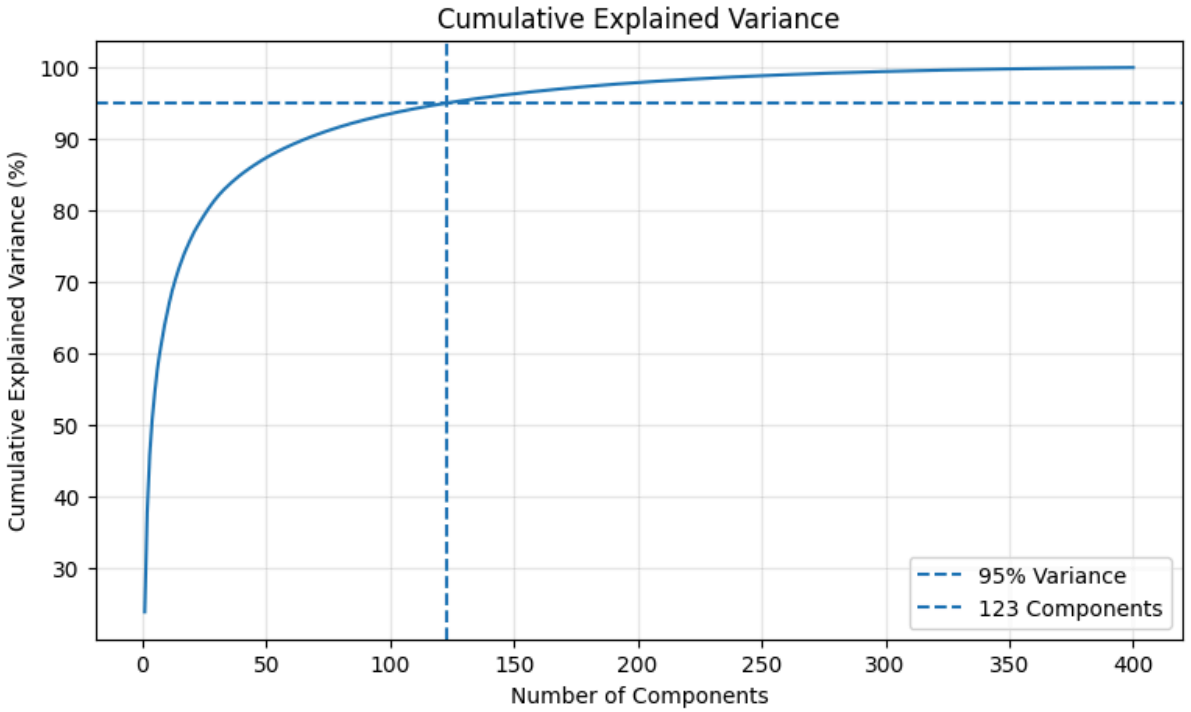
PCA From Scratch Completed
Eigenvalues Shape: (400,)
Eigenfaces Shape: (4096, 400)

Explained Variance of First 10 Components:
PC1: 23.8127%
PC2: 13.9940%
PC3: 7.9686%
PC4: 4.9983%
PC5: 3.6098%
PC6: 3.1569%
PC7: 2.4268%

PC8: 2.0364%
PC9: 1.9581%
PC10: 1.6721%

Components Required for 95% Variance: 123
Cumulative Variance at 95% Threshold: 95.03894 %



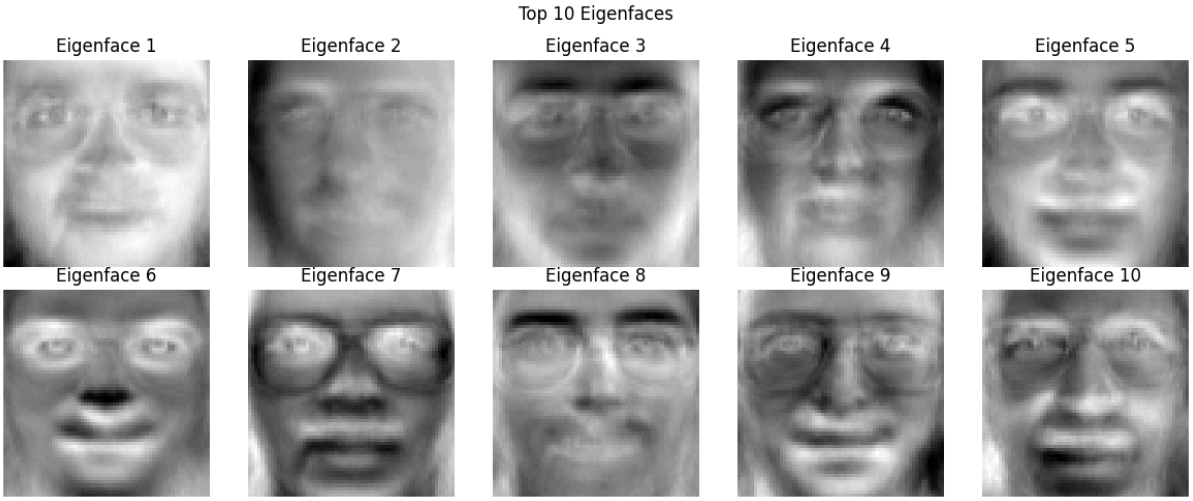


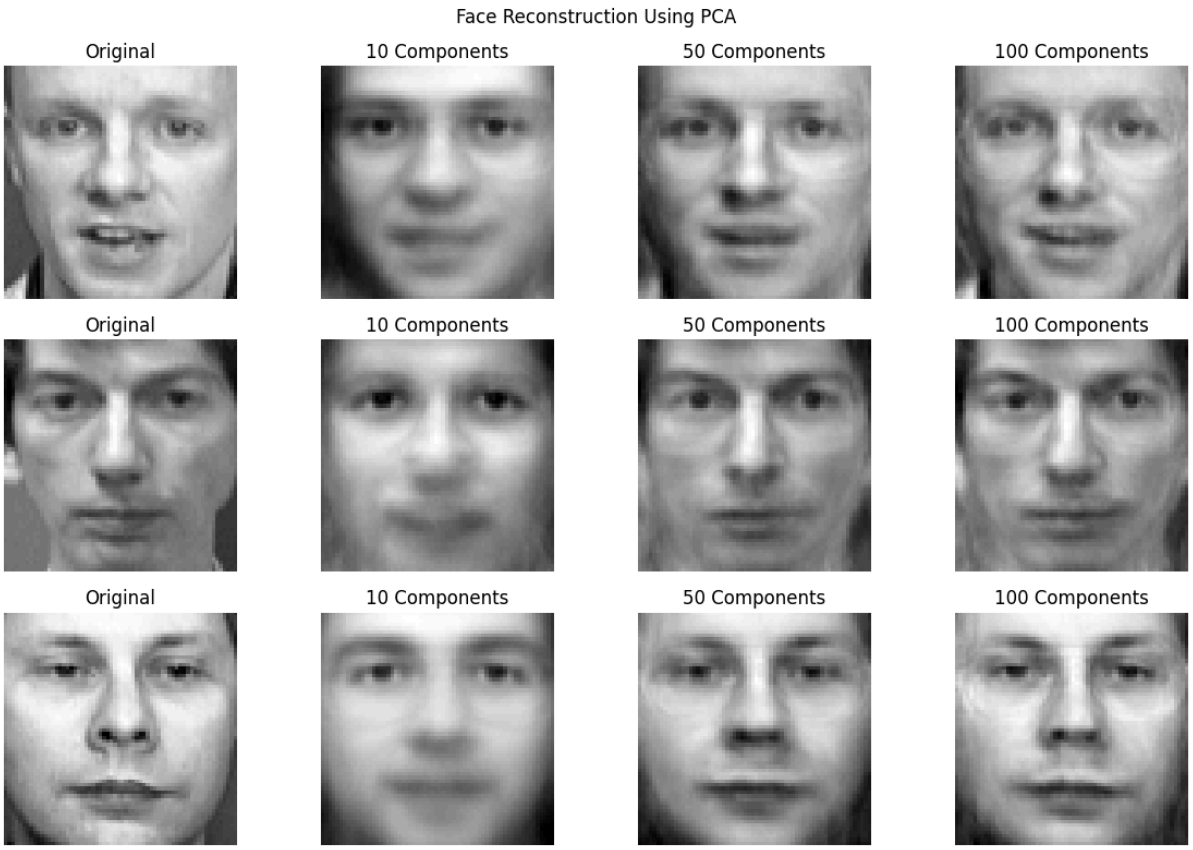
SCIKIT-LEARN PCA RESULTS
First 10 Eigenvalues:
[18.840168 11.071753   6.304612   3.954582   2.856037   2.4977117
  1.9200633 1.611159   1.5492215 1.3229506]

First 10 Explained Variance Ratios:
[0.23812726 0.13993962 0.07968613 0.0499833 0.03609842 0.03156942
 0.02426833 0.02036398 0.01958114 0.01672122]

Maximum Eigenvalue Difference:
9.536743e-06

Mean Absolute Component Similarity:
0.99999976





RECONSTRUCTION COMPARISON

Components	Explained Variance (%)	Reconstruction MSE
10	65.633904	0.00662
50	87.380577	0.00243
100	93.535316	0.00125

EXPERIMENT 19 COMPLETED
Original Features: 4096
Components for 95% Variance: 123

PCA Successfully Reduced Image Dimensions!

5. RESULT

Principal Component Analysis (PCA) was implemented from scratch using NumPy to reduce the dimensionality of the Olivetti Faces dataset. The covariance-based eigen-decomposition approach was used through an equivalent smaller Gram matrix to calculate principal components efficiently. The results were verified against Scikit-learn's PCA implementation. Scree plots and cumulative explained variance graphs were generated to identify the minimum number of components required to retain 95% of the variance. Finally, eigenfaces and reconstructed face images using 10, 50, and 100 components were visualized. Thus, the experiment demonstrates how PCA reduces image dimensionality while preserving important facial information.